



Lecture 4: Logistic Regression

Instructor: Xiang Ren
USC CSCI 544 Applied NLP
Fall 2026

Project Team Formation

- **Project team formation due by Sep 3 (today) 11:59pm**
- Teams of 6 students
 - We expect to see 47 groups, based on the current enrollment of 280 students
 - Exception case: **two groups will be 5 students** – let us know if you prefer

https://inklab.usc.edu/cs544_f26/details/project/

Class Project - Proposal

Due on Sep 10 → each team submit one on Brightspace

5% of total grade

The project proposal (**1 page, single-space, 10pt font size**) should outline the type of project (research-focused or application-focused), and then answer the following questions clearly in a sentence and/or a few paragraphs each:

- What are you trying to do? Articulate your objectives using absolutely no jargon.
- How is it done today, and what are the limits of current practice?
- What is new in your approach and why do you think it will be successful?
- Who cares?
- If you are successful, what difference will it make?
- What are the risks? What could go wrong, and how will you pivot early on if that happens?
- How much will it cost? That is, what resources will you need in terms of time and computation? Are these reasonable for a semester and what access you have?
- Identify two milestones along the way to your finished project.

Quiz Logistics

- The quiz will be administered through Brightspace.
- An in-class access passcode will be provided at the start of the quiz.
- A dynamic QR code will be displayed simultaneously for attendance verification.
- Students must complete the QR-code check-in within the designated attendance window.
- Completion of the Brightspace quiz does not, by itself, satisfy the attendance requirement.
- Students who fail to complete the attendance verification will receive a final quiz score of zero, regardless of the score earned on the Brightspace quiz.

Lecture Outline

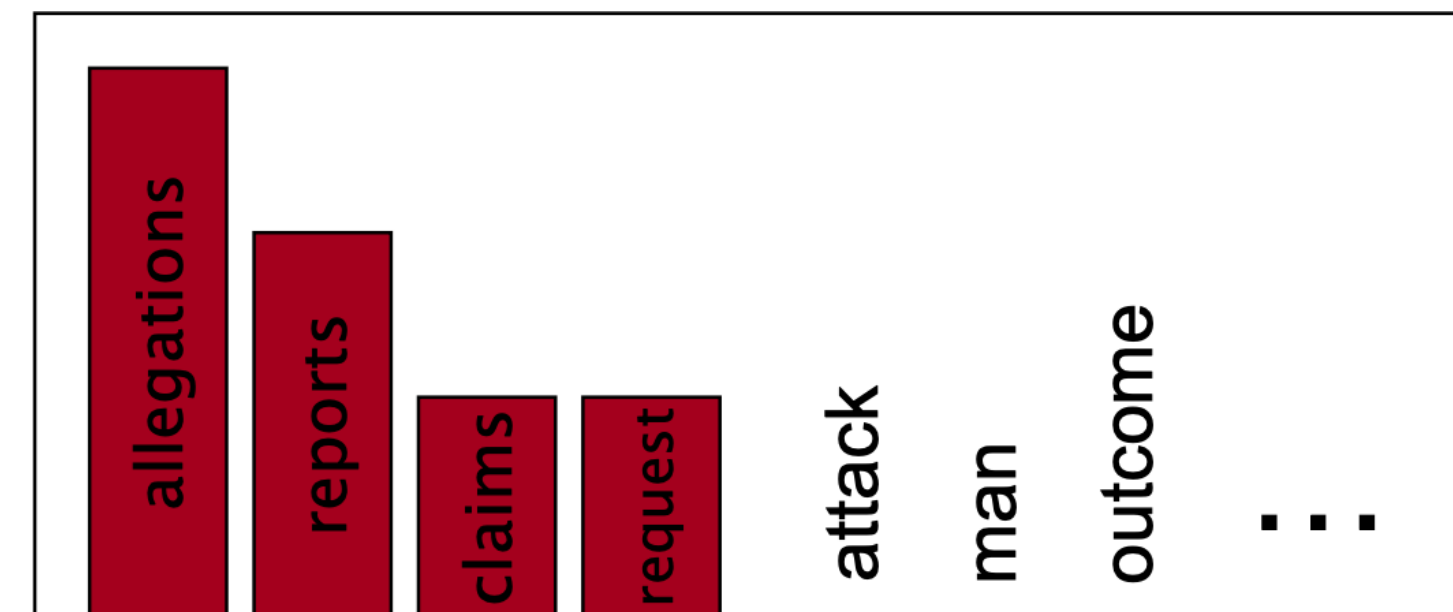
- Recap
 - Smoothing
 - Basics of Supervised Machine Learning
 - Data: Preprocessing and Feature Extraction
- Basics of Supervised Machine Learning
 - I. Data: Preprocessing and Feature Extraction
 - II. Model:
 - I. Logistic Regression
 - III. Loss
 - IV. Optimization Algorithm
 - V. Inference

Recap: Smoothing

Smoothing ~ Massaging Probability Masses

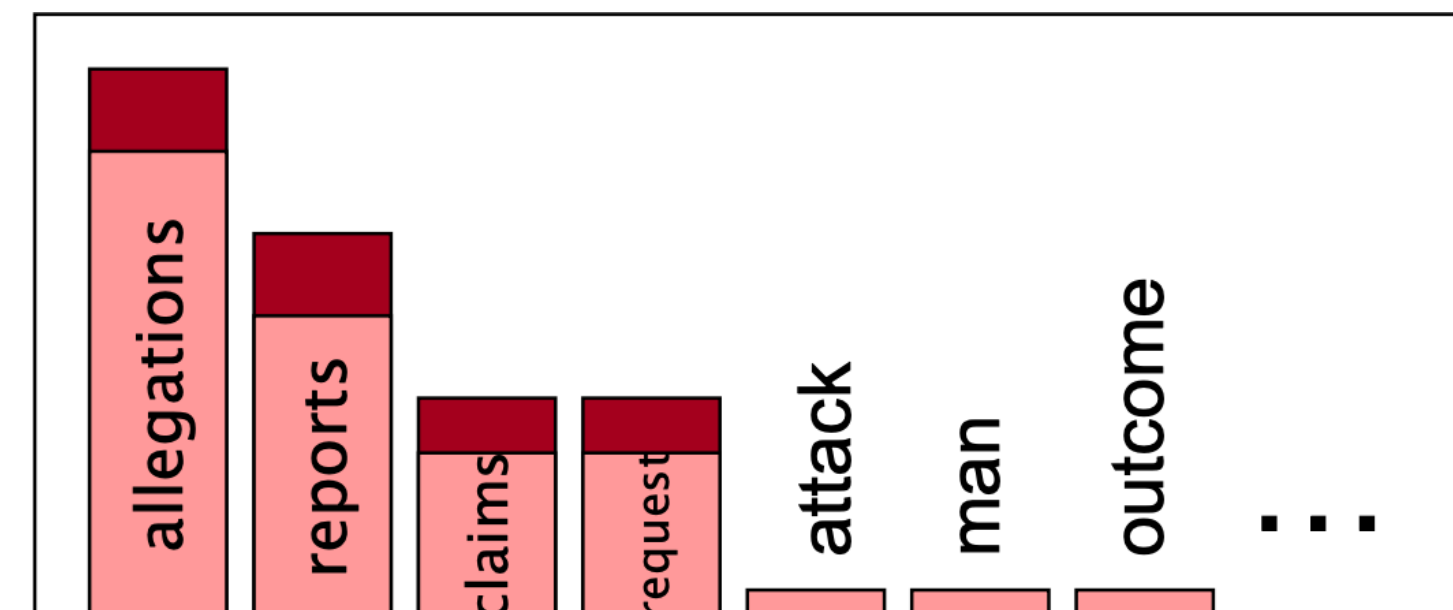
When we have sparse statistics: $Count(w \mid \text{denied the})$

3 allegations
2 reports
1 claims
1 request
7 total



Steal probability mass to generalize better: $Count(w \mid \text{denied the})$

2.5 allegations
1.5 reports
0.5 claims
0.5 request
2 other
7 total



Add-One Estimation

Laplace smoothing

1. Pretend we saw each n-gram one more time than we did
2. Just add one to all the n-gram counts!
3. All the counts that used to be zero will now have a count of 1...

Add-1 estimate for
Unigrams

$$P_{Add-1}(w_i) = \frac{c(w_i) + 1}{\sum_w (c(w) + 1)} = \frac{c(w_i) + 1}{V + \sum_w c(w)}$$

Add-One Estimation

Laplace smoothing

1. Pretend we saw each n-gram one more time than we did
2. Just add one to all the n-gram counts!
3. All the counts that used to be zero will now have a count of 1...

Add-1 estimate for
Unigrams

$$P_{Add-1}(w_i) = \frac{c(w_i) + 1}{\sum_w (c(w) + 1)} = \frac{c(w_i) + 1}{V + \sum_w c(w)}$$

Add-1 estimate for
Bigrams

$$P_{Add-1}(w_i | w_{i-1}) = \frac{c(w_{i-1}w_i) + 1}{c(w_{i-1}) + V}$$

Compare with raw bigram counts

Original, Raw

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Big change
to the counts!

Perhaps 1 is too
much, add a
fraction?

Reconstructed

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

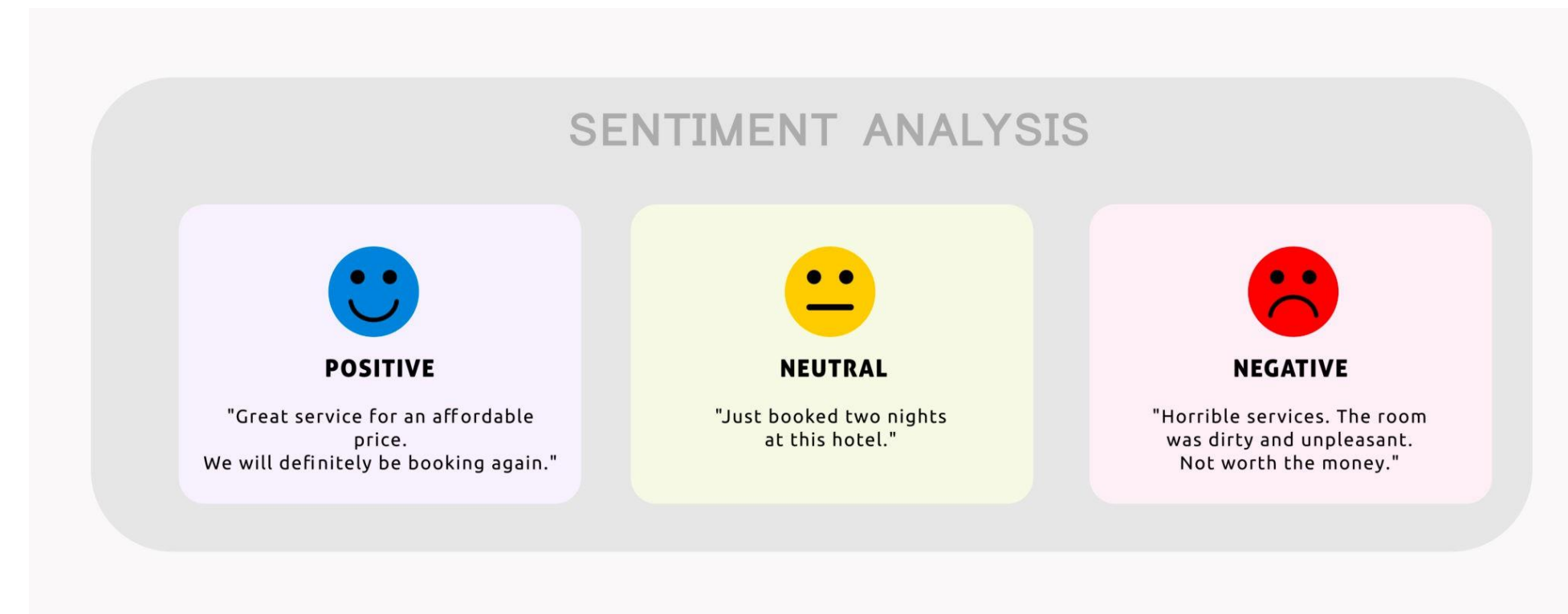
Add- k smoothing

k is a
hyperparameter

Recap: Basics of Supervised Machine Learning

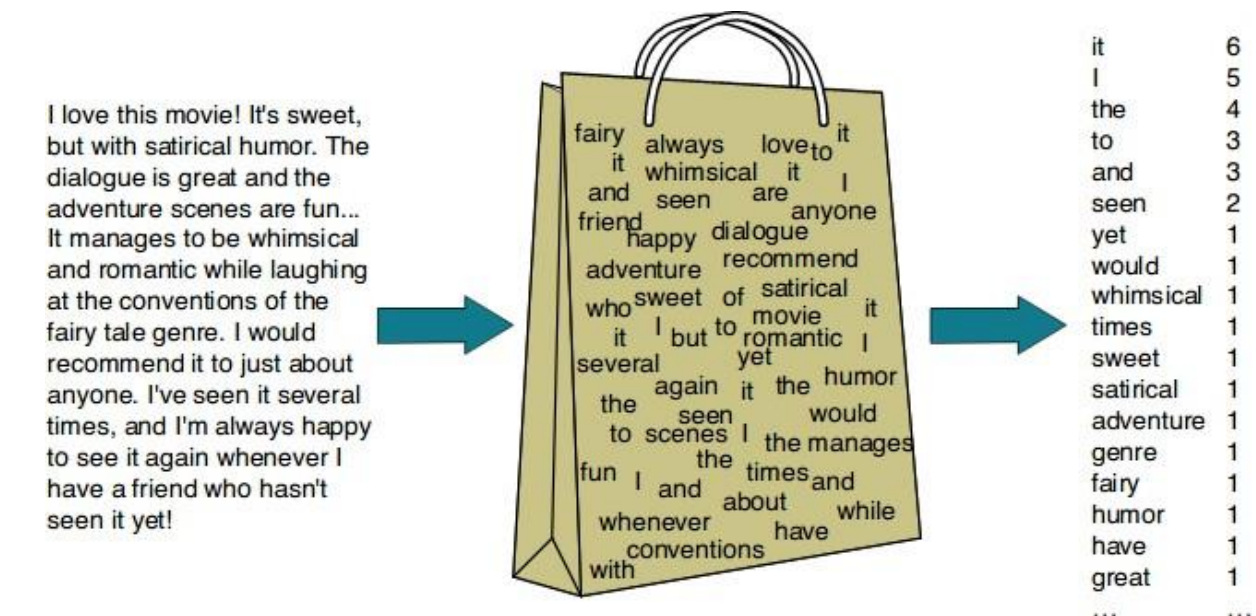
Features in Classification

- Examples of feature x_i
 - x_i = "review contains 'awesome'"; $w_i = +10$
 - x_j = "review contains 'abysmal'"; $w_j = -10$
 - x_k = "review contains 'mediocre'"; $w_k = -2$
- Each x_i is associated with a weight w_i which determines how important x_i is
 - (For predicting the positive class)
- May be
 - manually configured or
 - automatically inferred, as in modern architectures



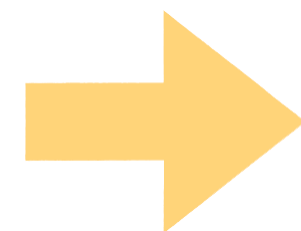
Another type of feature representation: Bag of Words

- With a word vocabulary of k words, BoW represents each doc (e.g., review) into a vector of integers
 - You may choose which k words, depending on the application
- $\mathbf{x} = [x_1, \dots, x_k]$, $x_i \in 0, 1, 2, \dots$
 - $x_i = j$ indicates that word i appears j times in the doc (e.g., review)



Feature Definition = [good, bad, nice, ugly, love, hate, complements, coarse, itchy]

"I love this shirt because it is nice and warm. The fabric is also nice and the color complements my skin tone."



$\mathbf{x} = [0, 0, 2, 0, 1, 0, 1, 0, 0]$

Bag of Words: Pros and Cons



- Limitations:
 - Insensitive to language structure: all **contextual information** has been discarded
 - Information in word dependencies is overlooked: **new york** vs **new book**
 - The resulting vectors are highly sparse
 - Dominated by **common words**
- Pros:
 - Simple!
 - Leads to acceptable performance in quite a few settings

Lecture Outline

- Recap
 - Smoothing
 - Basics of Supervised Machine Learning
 - Data: Preprocessing and Feature Extraction
- Basics of Supervised Machine Learning
 - I. Data: Preprocessing and Feature Extraction
 - II. Model:
 - I. Logistic Regression
 - III. Loss
 - IV. Optimization Algorithm
 - V. Inference

II. Model: Logistic Regression

Ingredients of Supervised Machine Learning

- I. Data as pairs $(x^{(i)}, y^{(i)})$ s.t $i \in \{1 \dots N\}$
 - $x^{(i)}$ usually represented by a feature vector $\mathbf{x}^{(i)} = [x_1, x_2, \dots, x_d]$,
e.g. word embeddings
- II. Model
 - A classification function that computes $\hat{y}$, the estimated class, via $p(y | x)$ e.g. logistic regression, naïve Bayes
- III. Loss

An objective function for learning

 - e.g. cross-entropy loss, L_{CE}
- IV. Optimization
 - An algorithm for optimizing the objective function
e.g. stochastic gradient descent
- V. Inference / Evaluation

Learning Phase

How to get the right y ?

- For each feature x_i , introduce a weight w_i , which determines the importance of x_i
 - Sometimes we have a bias term, b or w_0 , which is just another weight not associated to any feature
 - Together, all parameters can be termed as $\theta = [w; b]$
- We consider the weighted sum of all features and the bias

$$z = \left(\sum_d w_d x_d + b \right)$$
$$= \mathbf{w} \cdot \mathbf{x} + b$$

If high, $\hat{y} = 1$

If low, $\hat{y} = 0$

How to determine the threshold?

We need probabilistic models!

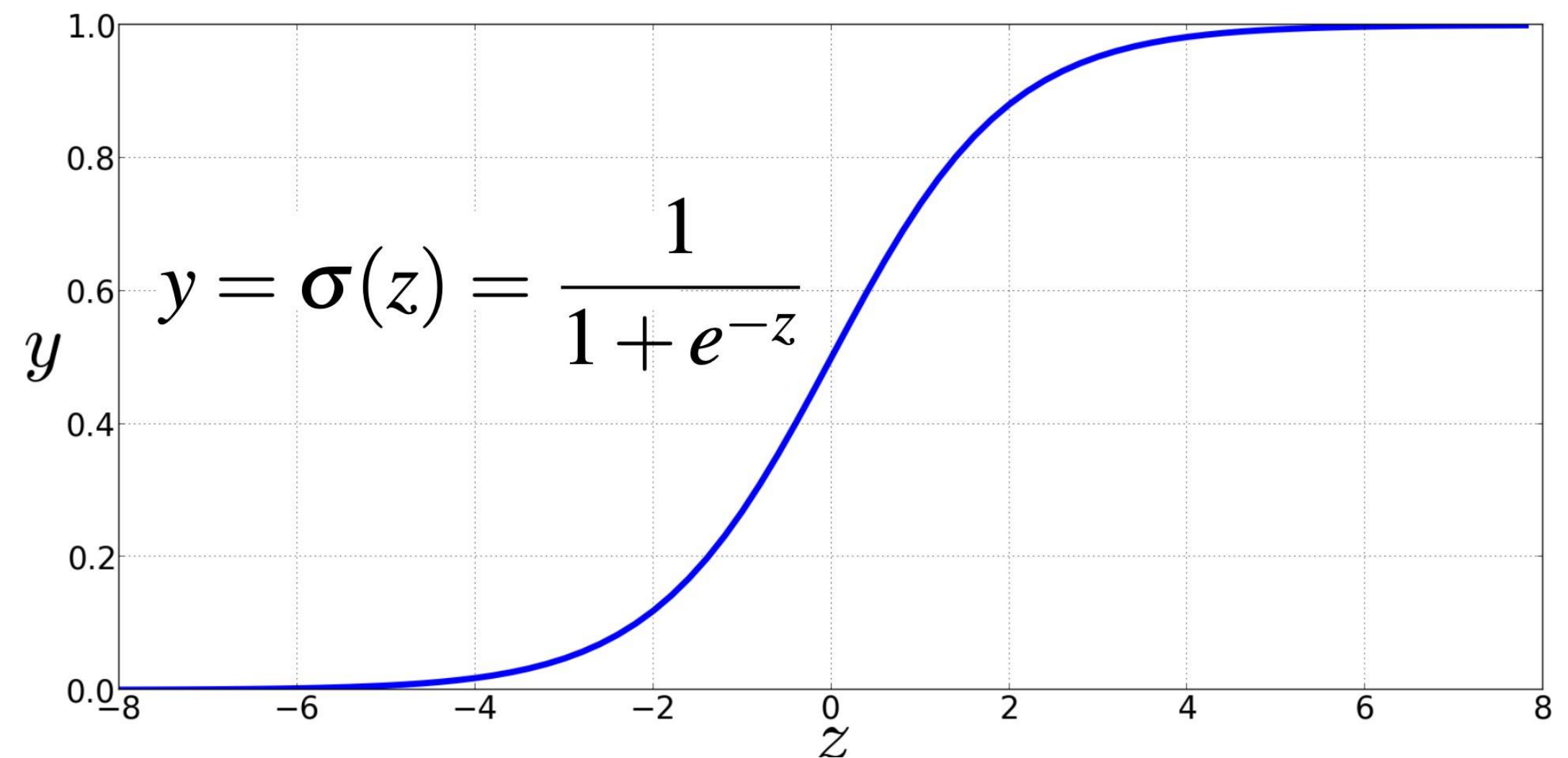
$$P(y = 1 \mid \mathbf{x}; \theta)$$

$$P(y = 0 \mid \mathbf{x}; \theta)$$

Solution: Squish it into the 0-1 range

$$z = \mathbf{w} \cdot \mathbf{x} + b \quad z \in \mathbb{R}$$

- Sigmoid Function, $\sigma(\cdot)$
 - Non-linear!
- Compute z and then pass it through the sigmoid function
- Treat it as a probability!
- Also, a differentiable function, which makes it a good candidate for optimization (more on this later!)



Sigmoids and Probabilities

$$\begin{aligned} P(y = 1|x; \theta) &= \sigma(w \cdot x + b) \\ &= \frac{1}{1 + \exp(-(w \cdot x + b))} \end{aligned}$$

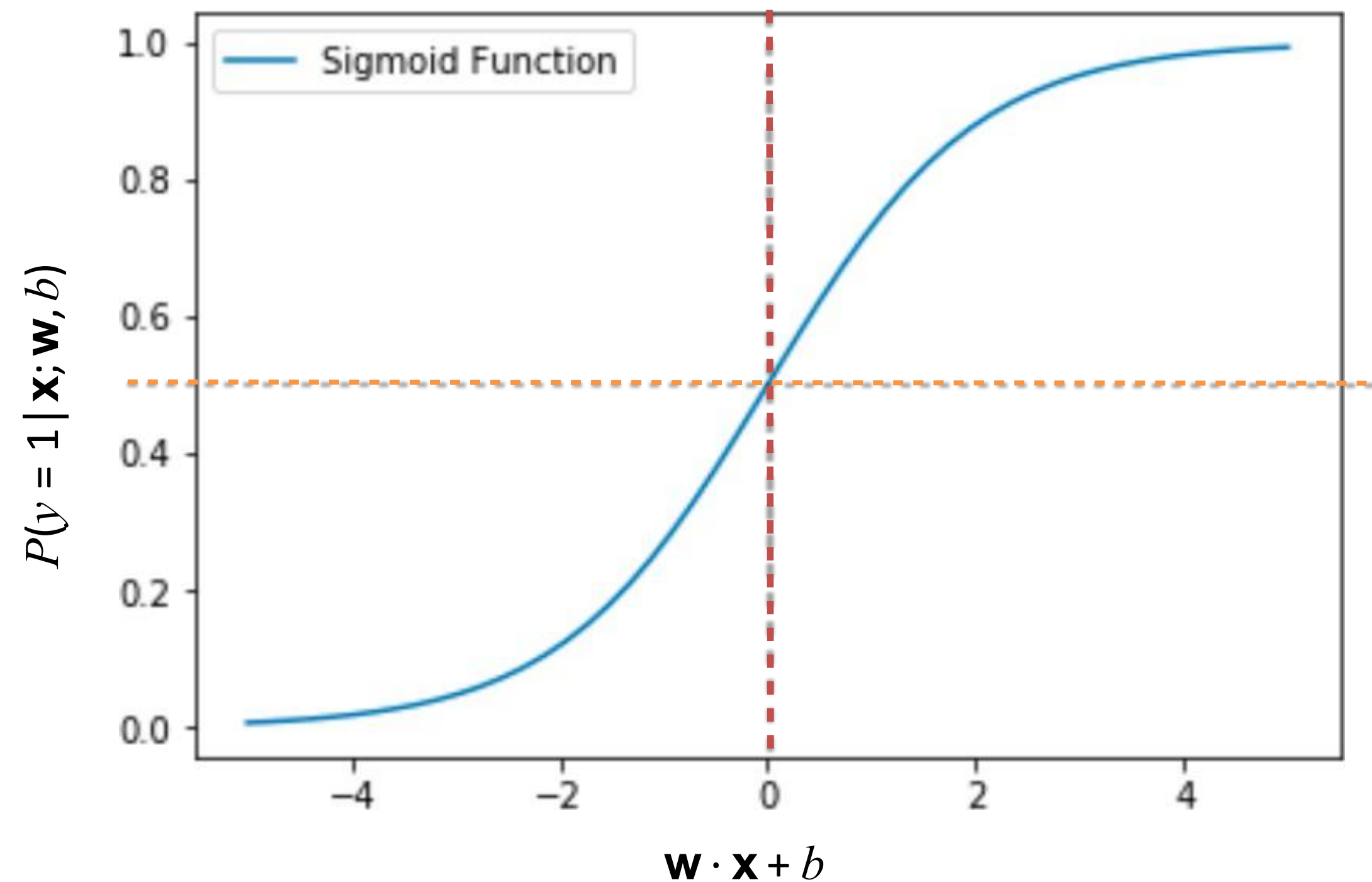
$$\begin{aligned} P(y = 0|x; \theta) &= 1 - \sigma(w \cdot x + b) \\ &= 1 - \frac{1}{1 + \exp(-(w \cdot x + b))} \\ &= \frac{\exp(-(w \cdot x + b))}{1 + \exp(-(w \cdot x + b))} \\ &= \frac{1}{1 + \exp(w \cdot x + b)} \\ &= \sigma(-(w \cdot x + b)) \end{aligned}$$

Classification Decision

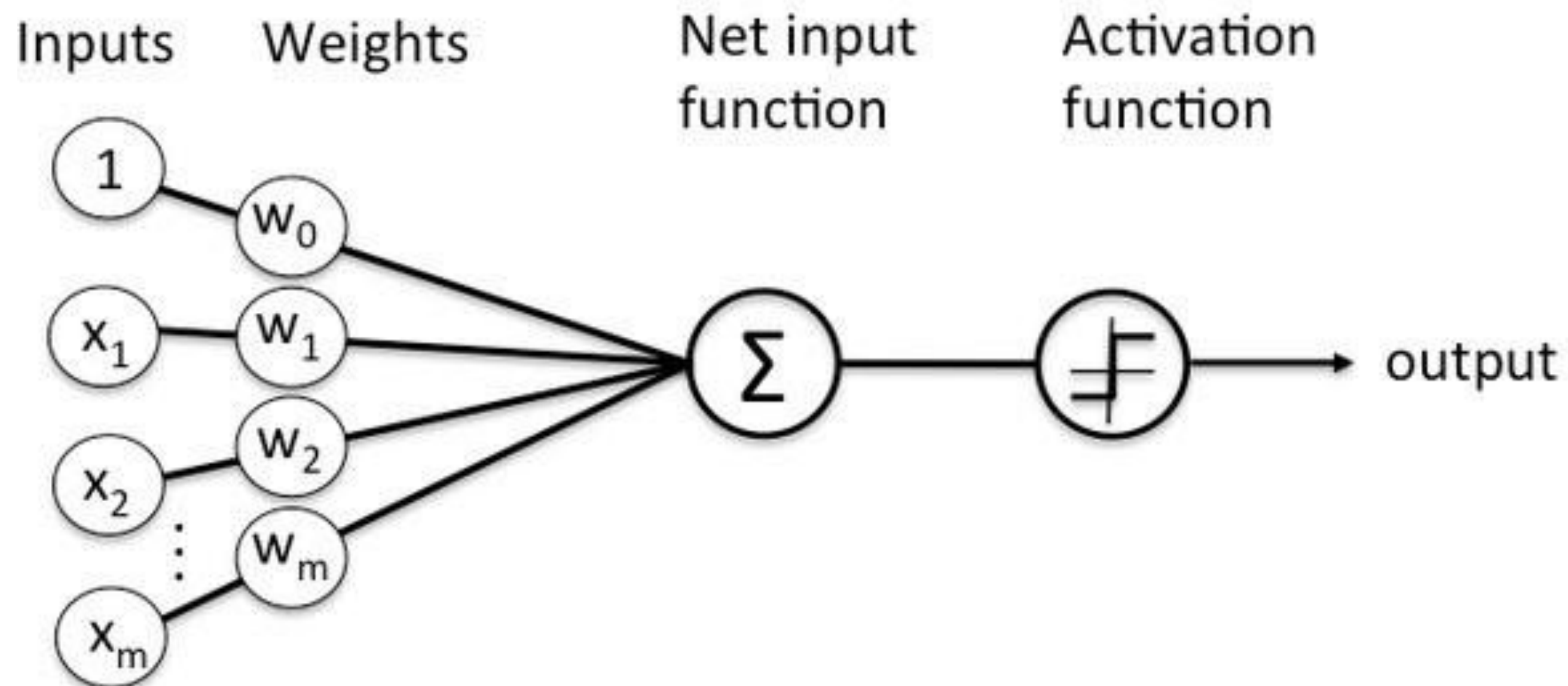
$$\hat{y} = \begin{cases} 1, & \text{if } p(y = 1|x; \theta) > 0.5. \\ 0, & \text{otherwise.} \end{cases}$$

decision boundary

$$\hat{y} = \begin{cases} 1, & \text{if } w \cdot x + b > 0. \\ 0, & \text{if } w \cdot x + b \leq 0. \end{cases}$$



Another notation



But where do the w 's and the b 's come from?

Supervised Classification:

- We know the correct label y (either 0 or 1) for each x
- But what the system produces is an estimate,
- Set w and b to minimize the distance between our estimate $\hat{y}^{(i)}$ and the true $y^{(i)}$
 - We need a distance estimator: a loss function or a cost function
 - We need an optimization algorithm to update w and b to minimize the loss.

Loss function

Optimization
Algorithm

Lecture Outline

- Recap
 - Smoothing
 - Basics of Supervised Machine Learning
 - Data: Preprocessing and Feature Extraction
- Basics of Supervised Machine Learning
 - I. Data: Preprocessing and Feature Extraction
 - II. Model:
 - I. Logistic Regression
 - III. Loss ←
 - IV. Optimization Algorithm
 - V. Inference

III. Loss: Cross-Entropy

The distance between $\hat{y}$ and y

$$\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$

- We want to know how far is the classifier output from the true (ground truth / gold standard) label

$$y \in \{0,1\}$$

- This difference is called the loss or cost
 - $L(\hat{y}, y)$ = how much $\hat{y}$ differs from y
 - In other words, how much would you lose if you mispredicted
 - Or how much would it cost you to mispredict

Remember maximum likelihood?

- Here: conditional maximum likelihood estimation
- We choose the parameters $\mathbf{W}, b$ that maximize the log probability
 - of the true labels y in the training data
 - given the observations x

$$\max \log p(y | x)$$

Suppose we flip the coin four times and see (H, H, H, T).

What is p ?



$p = 3/4 = 0.75$ maximizes the probability of data sequence (H,H,H,T)

maximum likelihood estimate

Maximizing conditional likelihood

Goal: maximize probability of the correct label $p(y | x)$

Since there are only 2 discrete outcomes (0 or 1) we can express the probability $p(y | \mathbf{x})$ from our classifier (the thing we want to maximize) as

$$p(y | x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

	$\hat{y} = 0$	$\hat{y} = 1$
$y = 0$	1	0
$y = 1$	0	1

Maximizing conditional likelihood

Goal: maximize probability of the correct label $p(y | \mathbf{x})$

$$\text{Maximize: } p(y | x) = \hat{y}^y (1 - \hat{y})^{1-y}$$

Now take the log of both sides

$$\begin{aligned} \log p(y | x) &= \log(\hat{y}^y (1 - \hat{y})^{1-y}) \\ &= y \log \hat{y} + (1 - y) \log(1 - \hat{y}) \end{aligned}$$

Whatever values
maximize $\log p(y | x)$
will also maximize
 $p(y | x)$

Minimizing negative log likelihood

Goal: maximize probability of the correct label $p(y | \mathbf{x})$

Maximize:
$$\begin{aligned}\log p(y | x) &= \log(\hat{y}^y (1 - \hat{y})^{1-y}) \\ &= y \log \hat{y} + (1 - y) \log(1 - \hat{y})\end{aligned}$$

Now flip the sign for something to minimize (we minimize the loss / cost)

Minimize:
$$\begin{aligned}L_{CE}(y, \hat{y}) &= -\log p(y | x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1 - y) \log \sigma(-(\mathbf{w} \cdot \mathbf{x} + b))]\end{aligned}$$

Measures how well the training data matches the proposed model distribution and how good the model distribution is

Cross-Entropy
Loss

Lecture Outline

- Recap
 - Smoothing
 - Basics of Supervised Machine Learning
 - Data: Preprocessing and Feature Extraction
- Basics of Supervised Machine Learning
 - I. Data: Preprocessing and Feature Extraction
 - II. Model:
 - I. Logistic Regression
 - III. Loss
 - IV. Optimization Algorithm ←
 - V. Inference

IV. Optimization: Stochastic Gradient Descent

Our goal: minimize the loss

- Loss function is parameterized by weights: $\theta = [\mathbf{w}; b]$ $\mathcal{L}_{CE}(f(x^{(i)}; \theta), y^{(i)})$
- We will represent $\hat{y}$ as $f(x; \theta)$ to make the dependence on θ more obvious

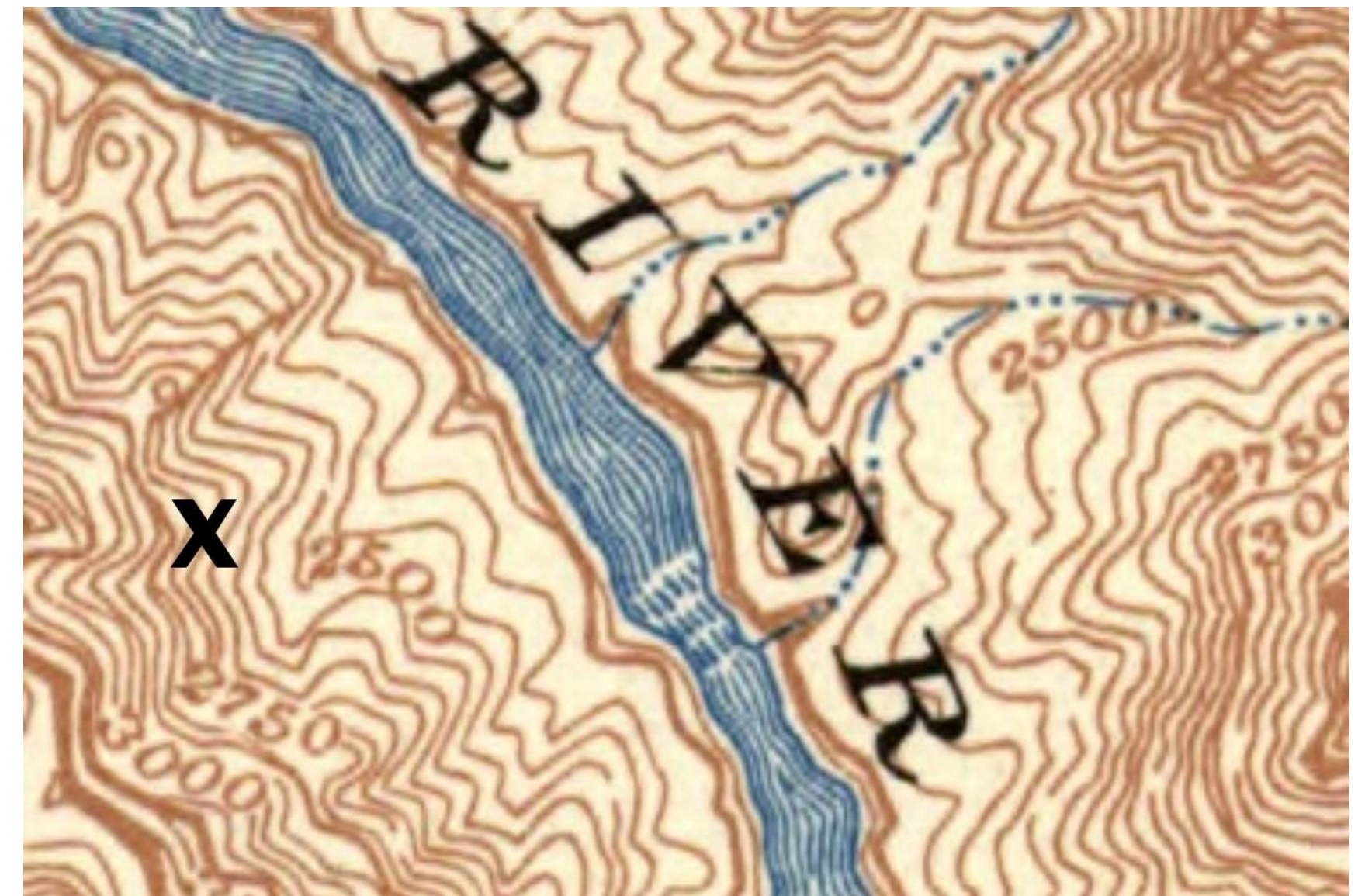
We want the weights that minimize the loss, averaged over all examples:

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m \mathcal{L}_{CE}(f(x^{(i)}; \theta), y^{(i)})$$

Intuition for gradient descent

How to get to the bottom of the river canyon?

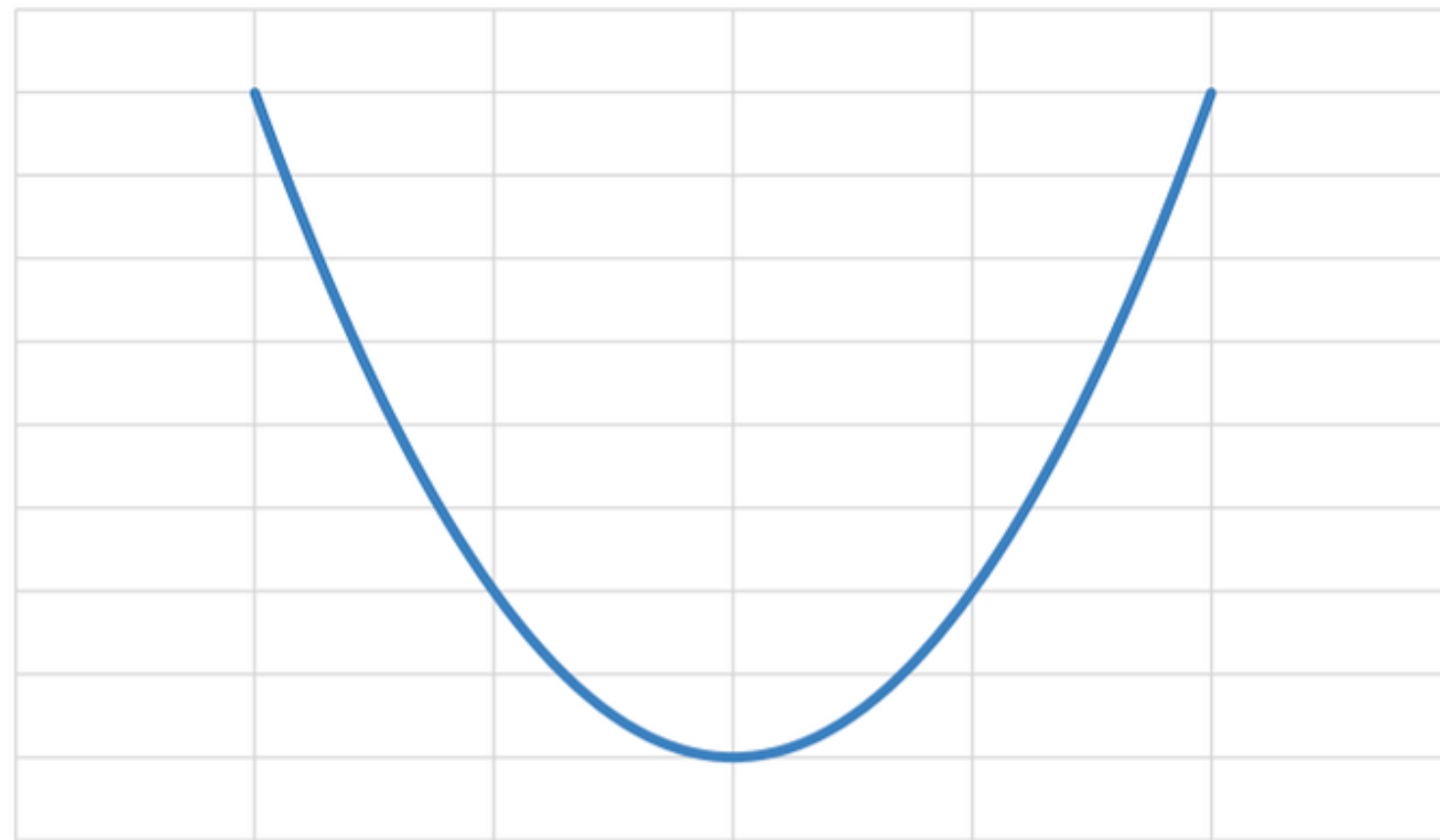
- Look around 360°
- Find the direction of steepest slope down
- Go that way



What if multiple equally good alternatives?

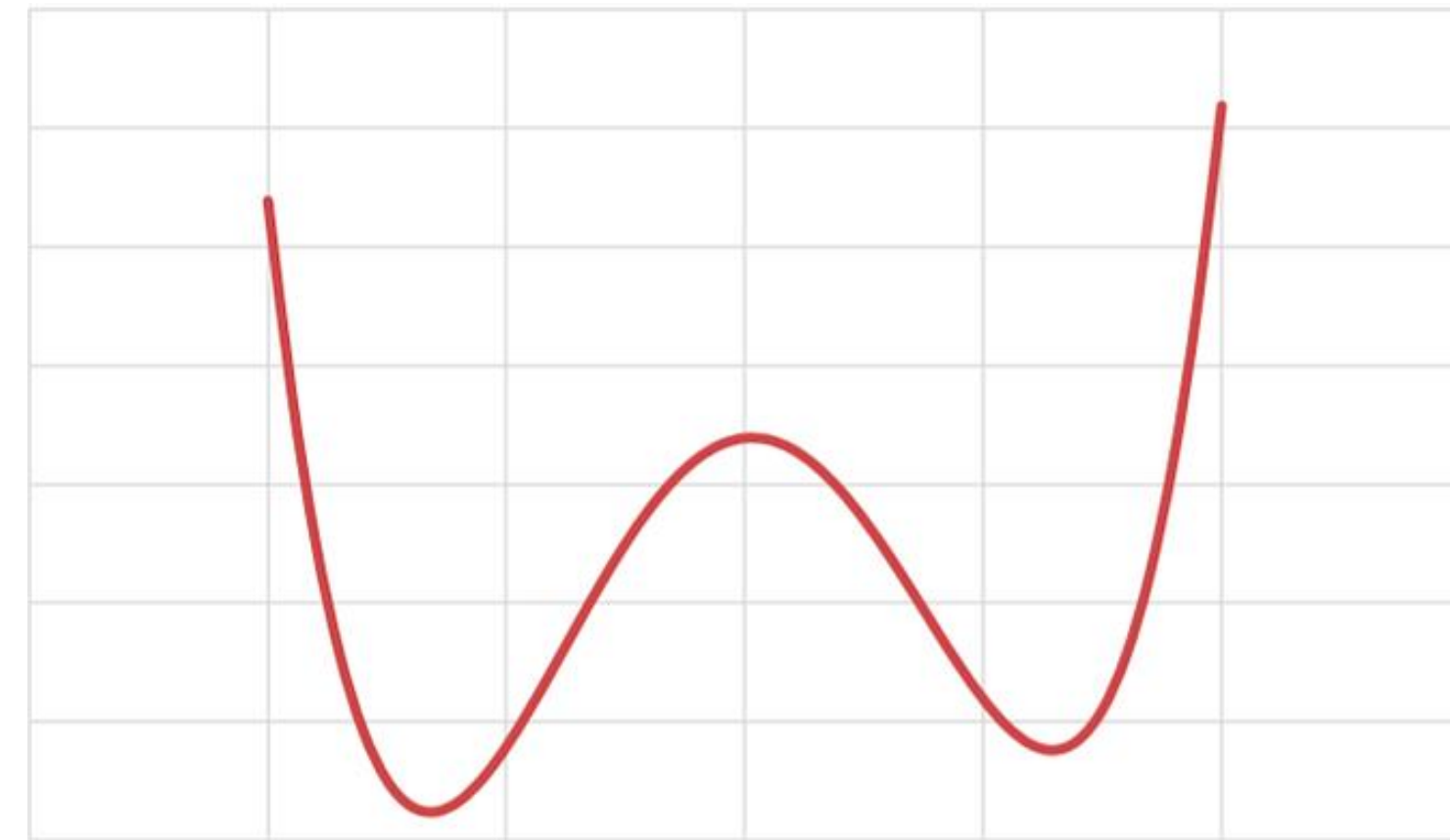
Logistic Regression: Loss

Image Credit: [Medium](#)



Convex function

- Has only one option for steepest gradient
 - Or one minimum
- Gradient descent starting from any point is guaranteed to find the minimum



Non-convex function

Neural Networks -
multiple alternatives

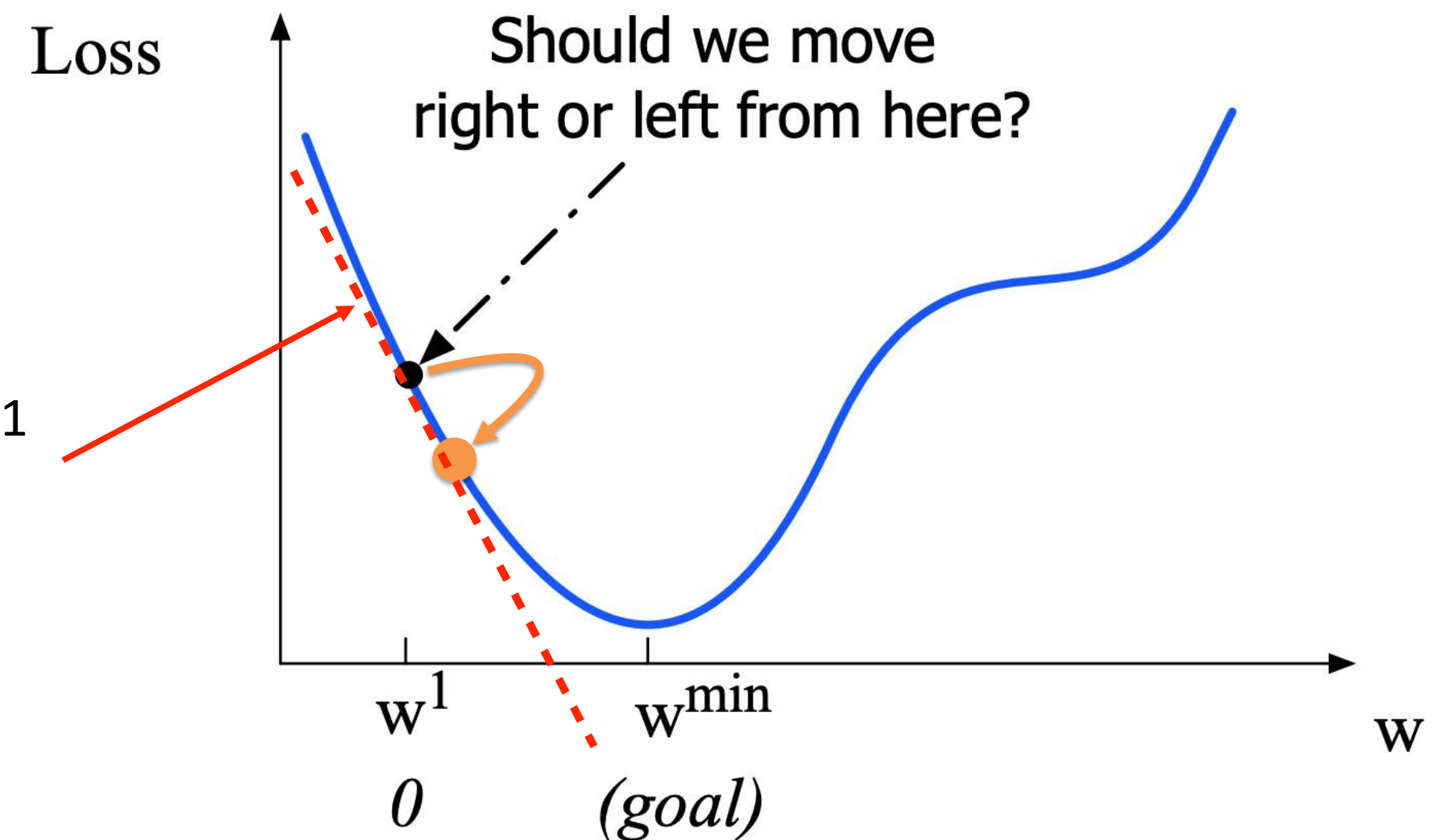
Consider: a single scalar

Given current w , should we make it bigger or smaller?

Move w in the reverse direction
from the slope of the function

slope of loss at w^1
is negative

need to move positive

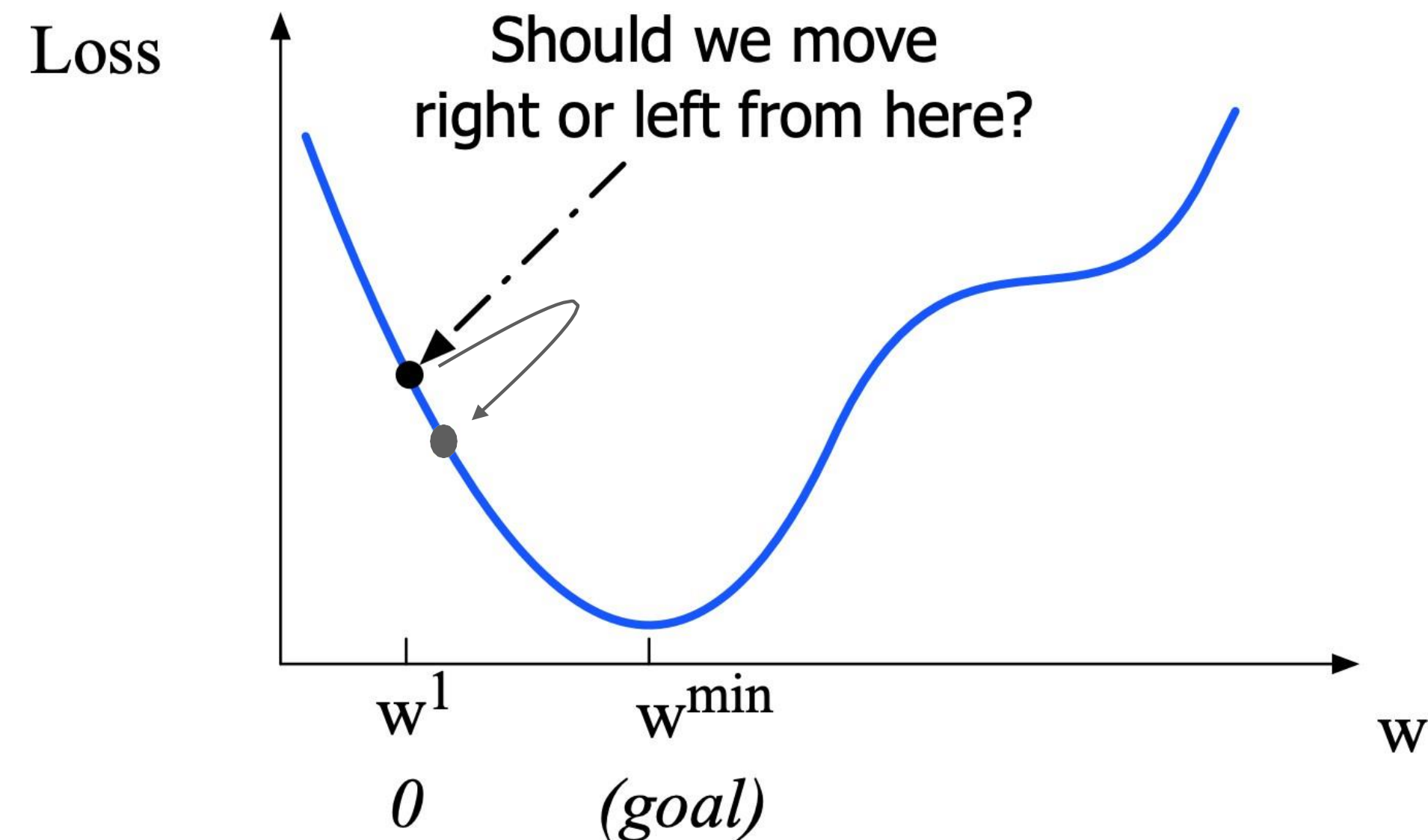


Gradients

The gradient of a function of many variables is a vector pointing in the direction of the greatest increase in a function.

Find the gradient of the loss function at the current point and move in the opposite direction.

Gradient Descent

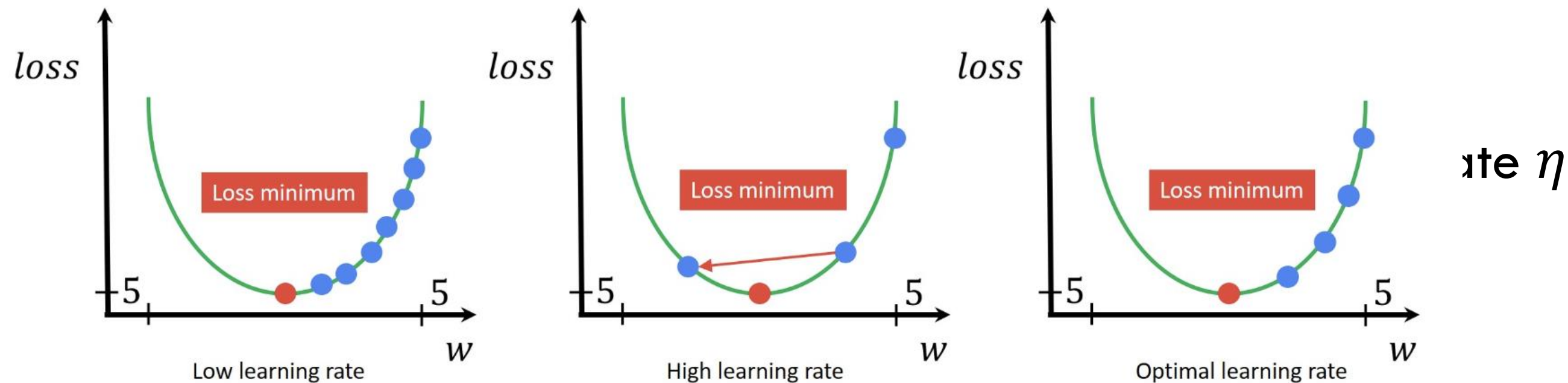


But by how much?

Gradient Updates

Move w by the

- Higher learning rate



η Too high: the learner will take big steps and overshoot

$$w_{t+1} = w_t - \eta \frac{\partial}{\partial w} L(f(x; w), y)$$

η Too low: the learner will take too long

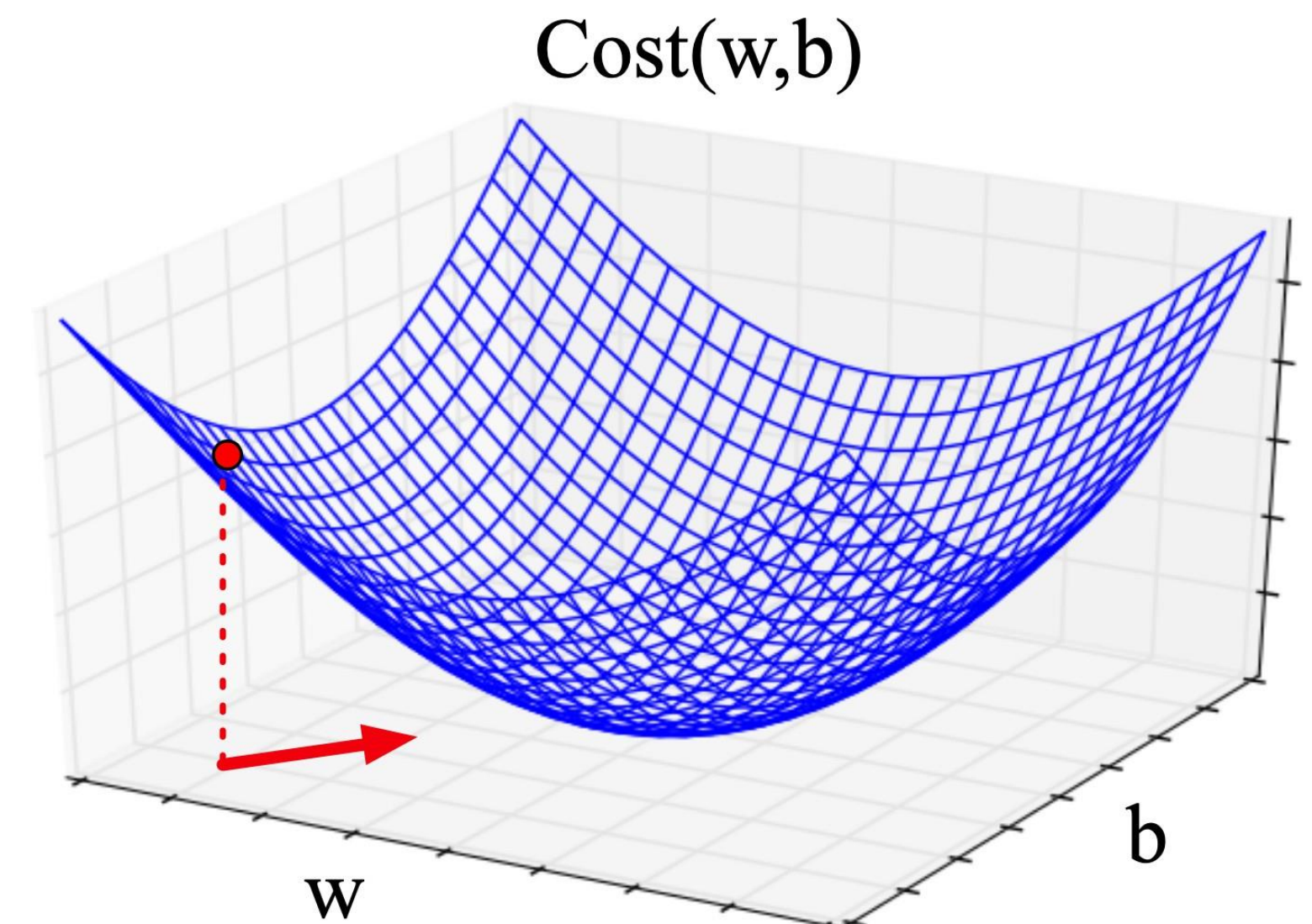
If parameter w is a vector of d dimensions:

The gradient is just such a vector; it expresses the directional components of the sharpest slope along each of the d dimensions.

Under 2 dimensions

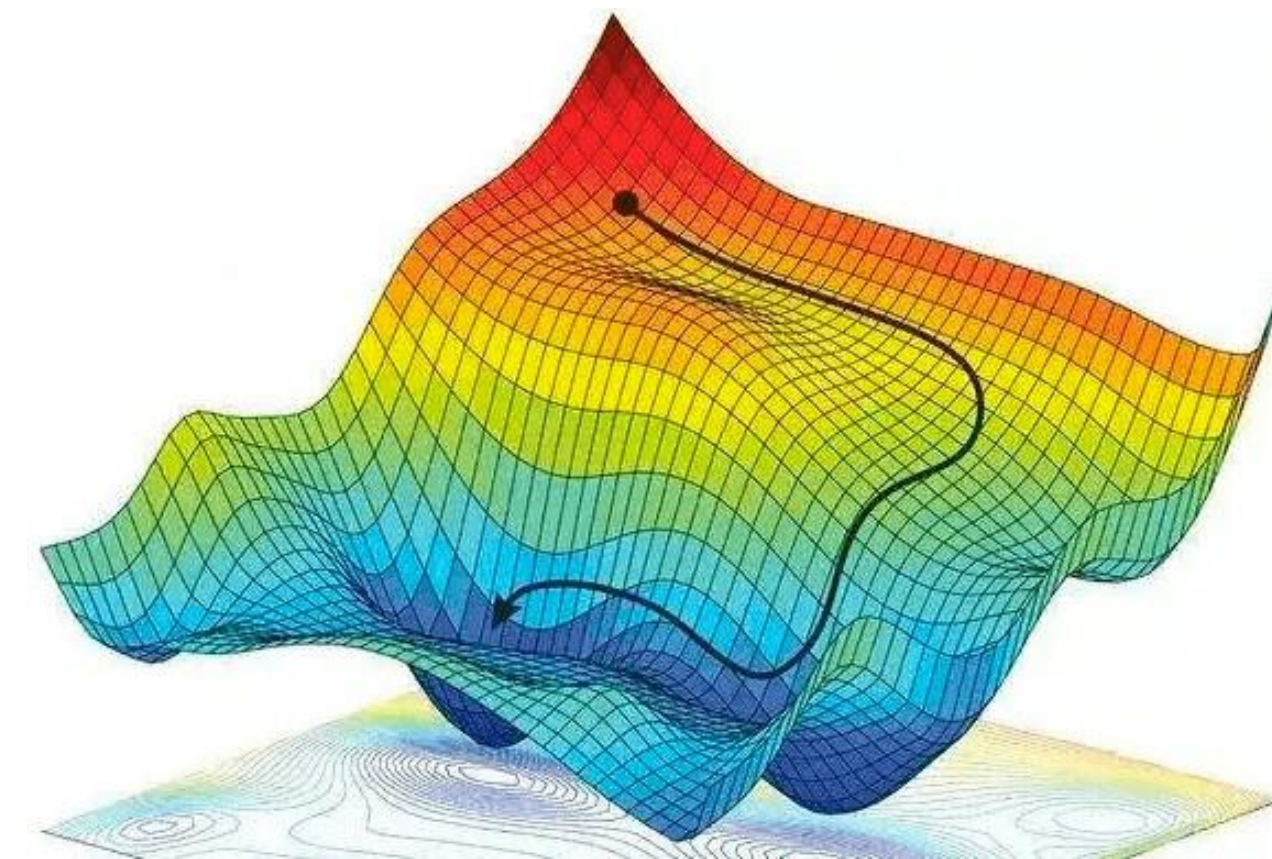
Consider 2 dimensions, w and b :

The gradient vector at the red point has two dimensions in the w - b plane



Real-life gradients, however...

- ...are much longer; models usually contain lots and lots of weights!
- For each dimension θ_i the gradient component tells us the slope with respect to that variable
 - “How much would a small change in θ_i influence the total loss function L ”
- We express the slope as a partial derivative of the loss
 - The gradient is then defined as a vector of these partials



Real-life gradients

$$\nabla_{\theta} \mathcal{L}(f(x; \theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} \mathcal{L}(f(x; \theta), y) \\ \frac{\partial}{\partial w_2} \mathcal{L}(f(x; \theta), y) \\ \dots \\ \frac{\partial}{\partial w_n} \mathcal{L}(f(x; \theta), y) \end{bmatrix}$$

The final equation for updating θ at time step $t + 1$ based on the gradient is thus:

$$\theta_{t+1} = \theta_t - \eta \frac{\partial}{\partial \theta} L(f(x; \theta), y)$$

Gradients for Logistic Regression

Case: Sentiment Analysis

Recall: the cross-entropy loss for logistic regression

$$L_{CE}(y, \hat{y}) = - [y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1 - y) \log(\sigma(-(\mathbf{w} \cdot \mathbf{x} + b)))]$$

Derivatives have a closed form solution:

$$\frac{\partial L_{CE}(y, \hat{y})}{\partial w_j} = [\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y]x_j$$

Pseudocode

function STOCHASTIC GRADIENT DESCENT ($L()$, $f()$, x , y) returns θ

where: L is the loss function

f is a function parameterized by θ

$\mathbf{x}$ is the set of training inputs $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}$

y is the set of training outputs (labels) $y^{(1)}, y^{(2)}, \dots, y^{(N)}$

$\theta \leftarrow 0$ (or randomly initialized)

repeat till done

for each training tuple $(x^{(i)}, y^{(i)})$: (in random order)

1. Compute $\hat{y}^{(i)} = f(\mathbf{x}^{(i)}; \theta)$

What is our estimated output $\hat{y}^{(i)}$?

2. Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$

How far off is $\hat{y}^{(i)}$ from the true label $y^{(i)}$?

3. $g \leftarrow \nabla L(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$

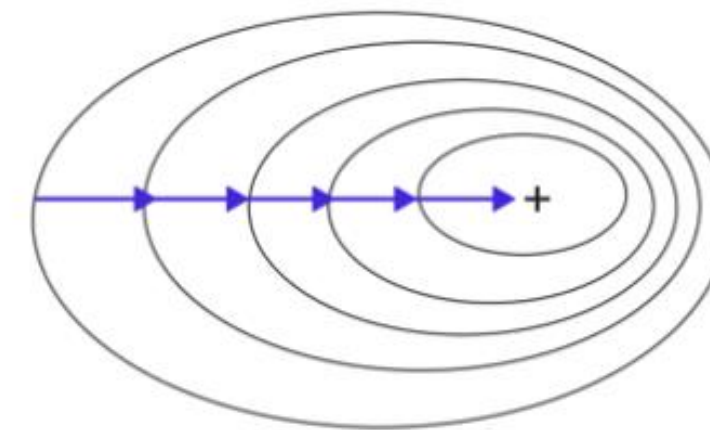
What is the direction that maximizes L ? (gradient)

4. $\theta \leftarrow \theta - \eta g$

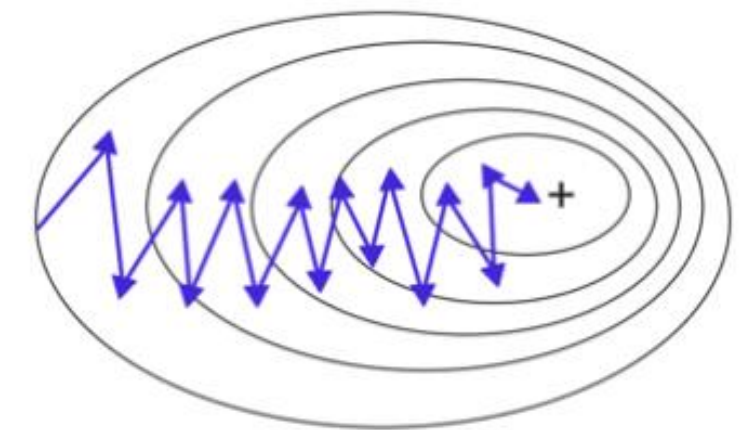
Go the other way instead

return θ

GD



SGD



Stochastic Gradient Descent

Mini-Batching

function STOCHASTIC GRADIENT DESCENT ($L()$, $f()$, x , y) returns θ

where: L is the loss function

f is a function parameterized by θ

$\mathbf{X}$ is the set of training inputs $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots, \mathbf{x}^{(N)}$

y is the set of training outputs (labels) $y^{(1)}, y^{(2)}, \dots, y^{(N)}$ and m is the mini-batch size

$\theta \leftarrow 0$ (or randomly initialized)

repeat till done

for each randomly sampled minibatch of size m :

1. for each training tuple $(\mathbf{x}^{(i)}, y^{(i)})$ in the minibatch: (in random order)

i. Compute $\hat{y}^{(i)} = f(\mathbf{x}^{(i)}; \theta)$

What is our estimated output $\hat{y}^{(i)}$?

ii. Compute the loss $L_{mini} \leftarrow L_{mini} + L(\hat{y}^{(i)}, y^{(i)})$

How far off is $\hat{y}^{(i)}$ from the true output $y^{(i)}$?

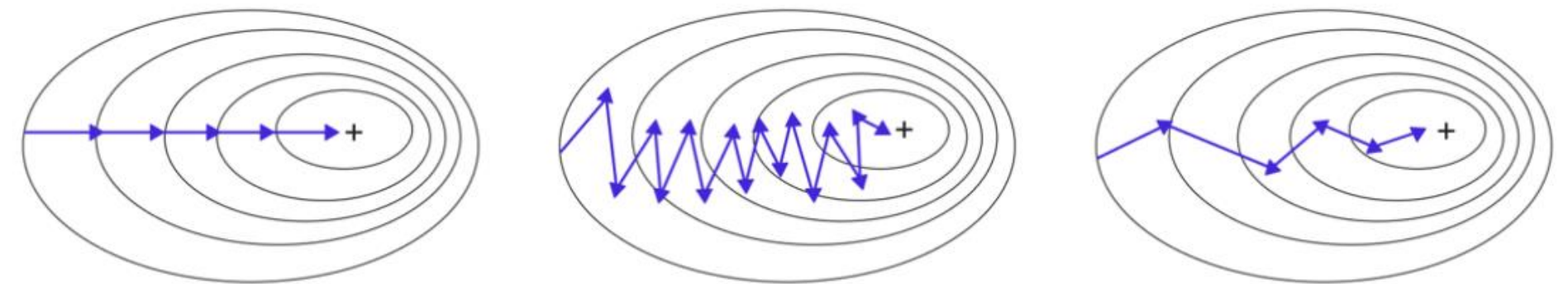
2. $g \leftarrow \frac{1}{m} \nabla L_{mini}(f(\mathbf{x}^{(i)}; \theta), y^{(i)})$

How should we move θ to maximize loss?

3. $\theta \leftarrow \theta - \eta g$

Go the other way instead

return θ



Regularization

Overfitting

- A model that perfectly match the training data has a problem
- It will also overfit to the data, modeling noise
 - A random word that perfectly predicts y (it happens to only occur in one class) will get a very high weight
 - Failing to generalize to a test set without this word

Why?

A good model should be able to generalize

What happens when a feature only occurs with one class?

e.g. word “wow” for positive reviews

Machine Learning Math Formulation

- Given training dataset D

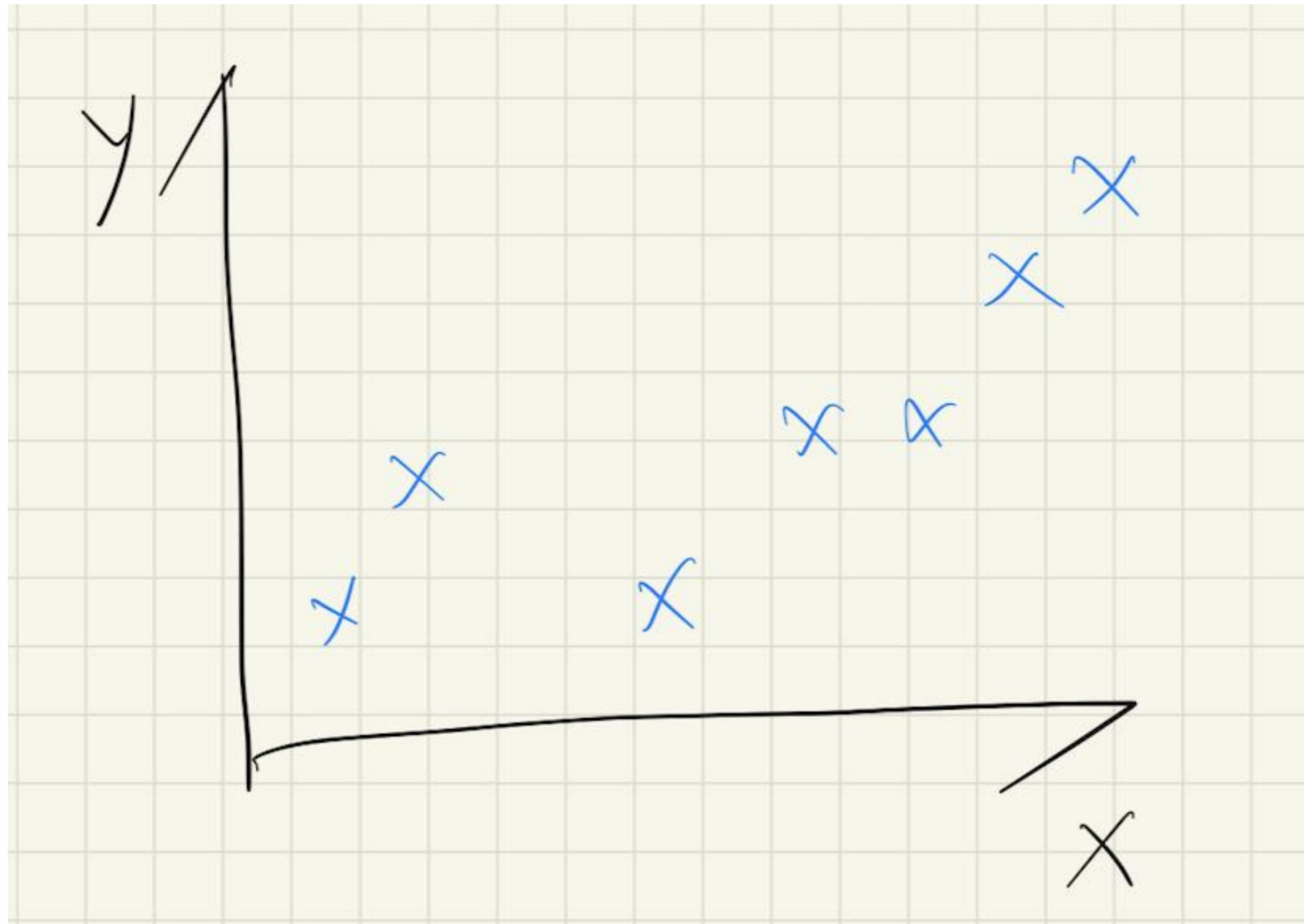
We have **features** and **labels** data (X, Y) :

$$\begin{array}{c} (x_1, y_1) \\ (x_2, y_2) \\ \vdots \\ (x_n, y_n) \end{array}$$

Features  **Actual Labels** 

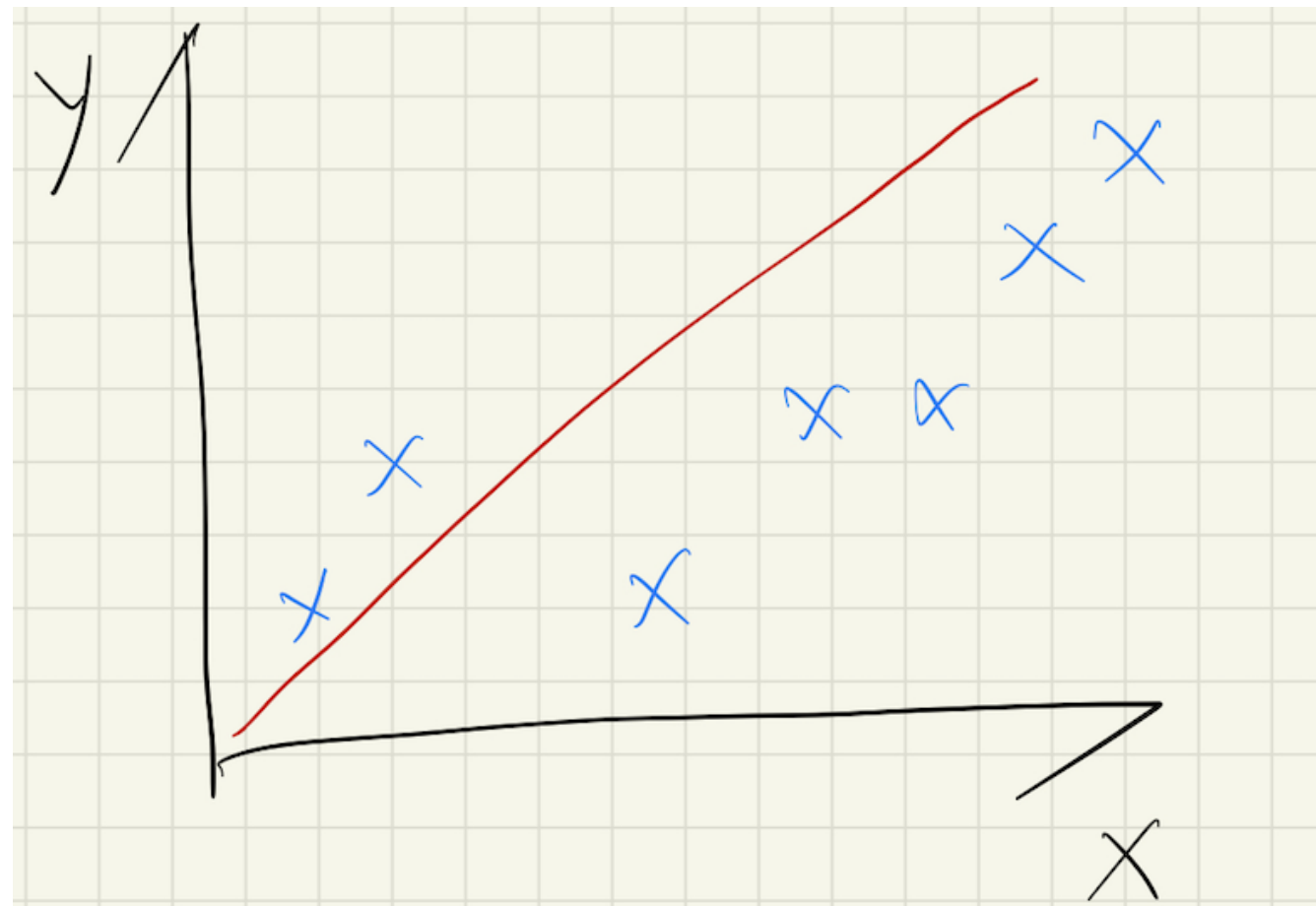
- Find a function $f(x)$ that can minimize a loss function

Learning: Good or Bad?



Learning: Good or Bad?

avoid this

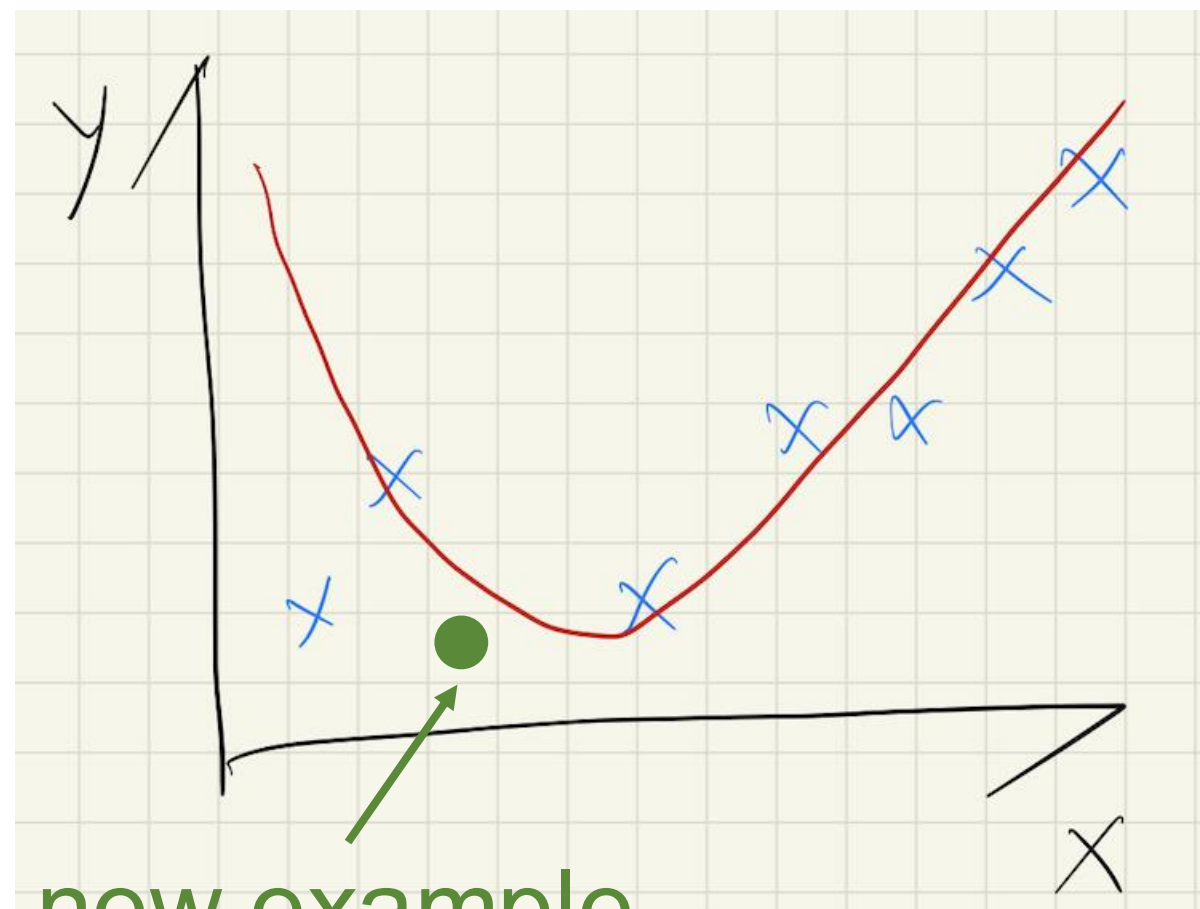


Features = $[1, x]$

underfitting

Too simple

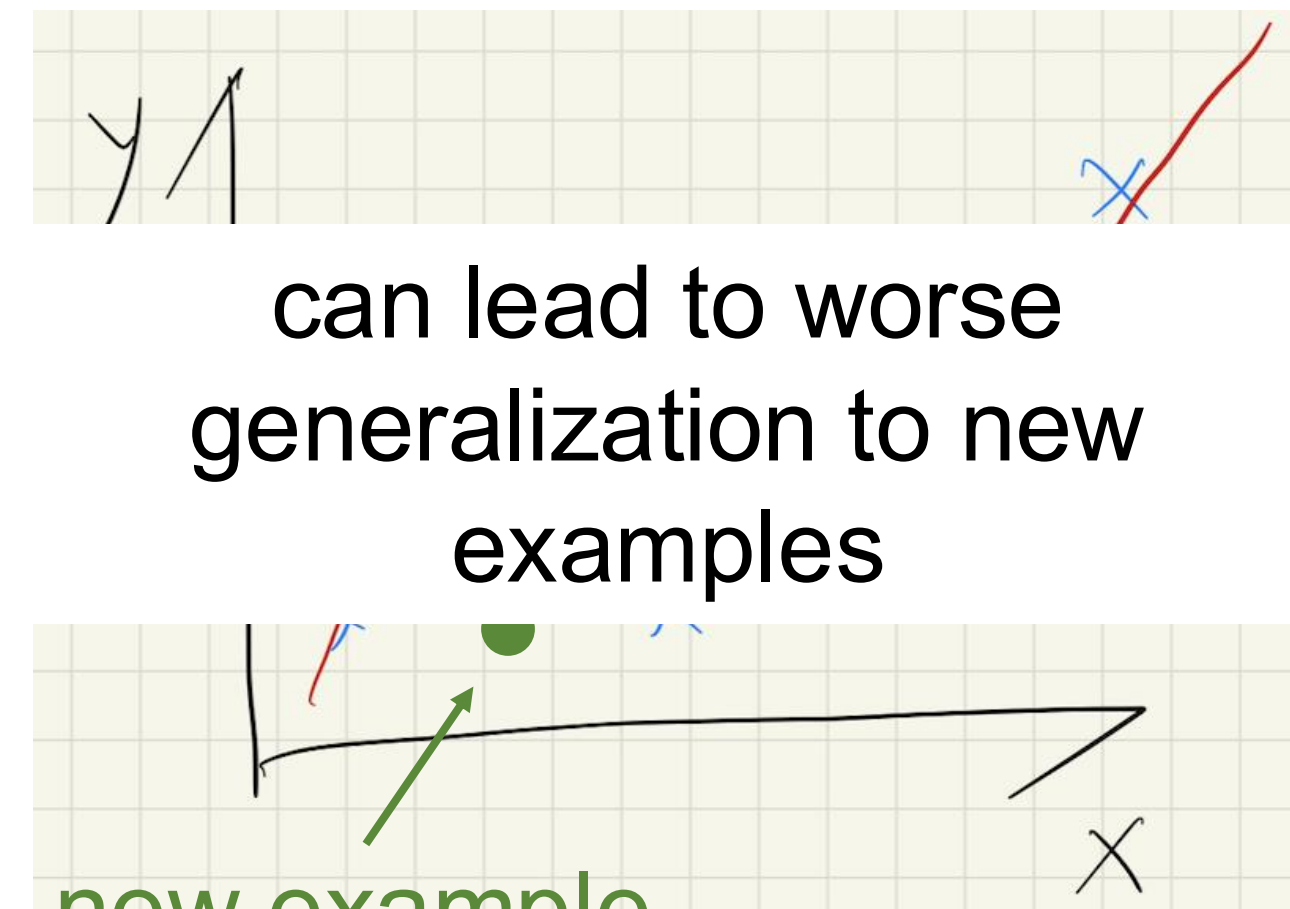
want this



Features = $[1, x, x^2]$

good balance b/w
underfitting and overfitting

avoid this



can lead to worse
generalization to new
examples

Features = $[1, x, x^2, \dots, x^7]$

0 training loss!

Learns a function that is too
complex

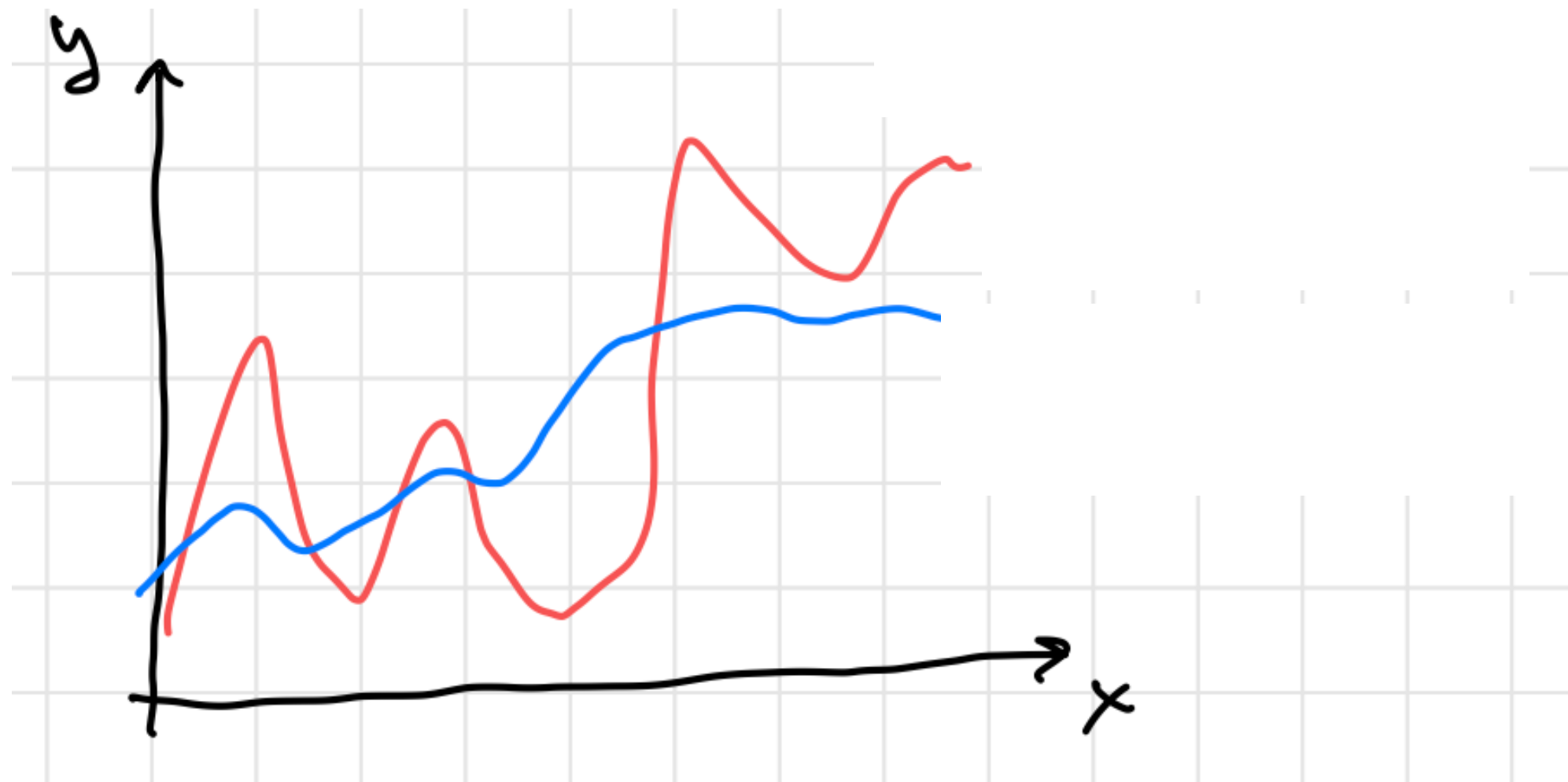
overfitting

Regularization

- A technique to reduce overfitting
 - by encouraging models to be “simpler”

L2 regularization

Encourage L2 norm of parameters to be small

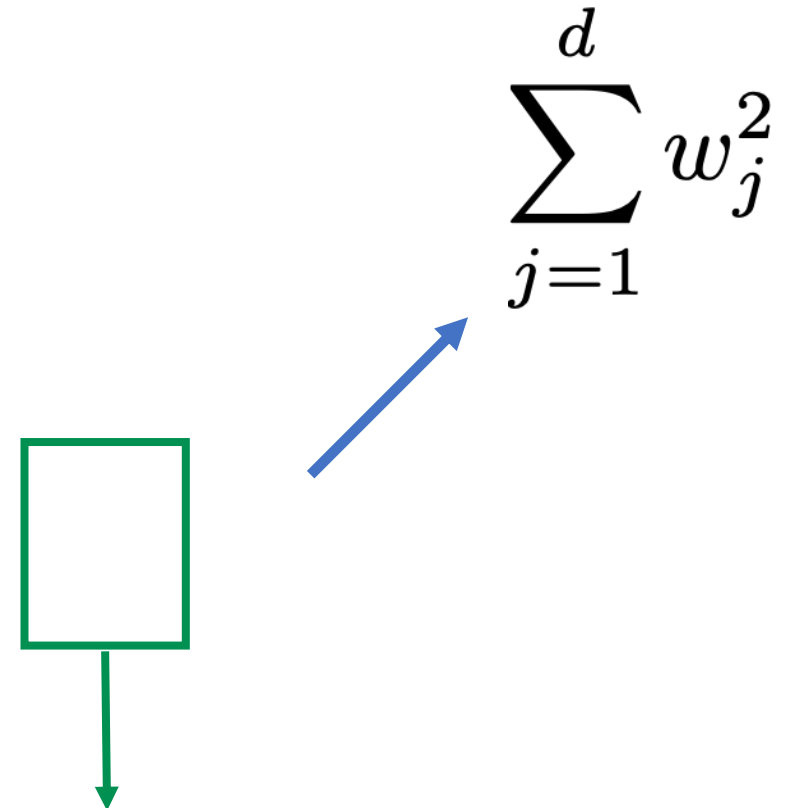


$$\|w\| = \sqrt{\sum_{j=1}^d w_j^2}$$

Regularization: How?

- Add term to loss function. e.g., for regression

original loss

$$L(w) = \frac{1}{n} \sum_{i=1}^n (w^\top x^{(i)} - y^{(i)})^2$$


new hyperparameter!

$\sum_{j=1}^d w_j^2$

can be any number ≥ 0

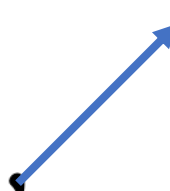
if $=0$, then no regularization

larger value $\rightarrow$ stronger regularization

Regularization: How?

$$L(w) = \frac{1}{n} \sum_{i=1}^n (w^\top x^{(i)} - y^{(i)})^2 + \lambda \|w\|^2$$

- How does this affect gradient?

$$\nabla_w L(w) = [\text{old gradient}] + \lambda \cdot 2w$$

$$\frac{d}{dw_j} \sum_{k=1}^d w_k^2 = 2 \cdot w_j$$

- Gradient Descent update: subtract multiple of w from w : i.e., to move towards origin (“weight decay”)

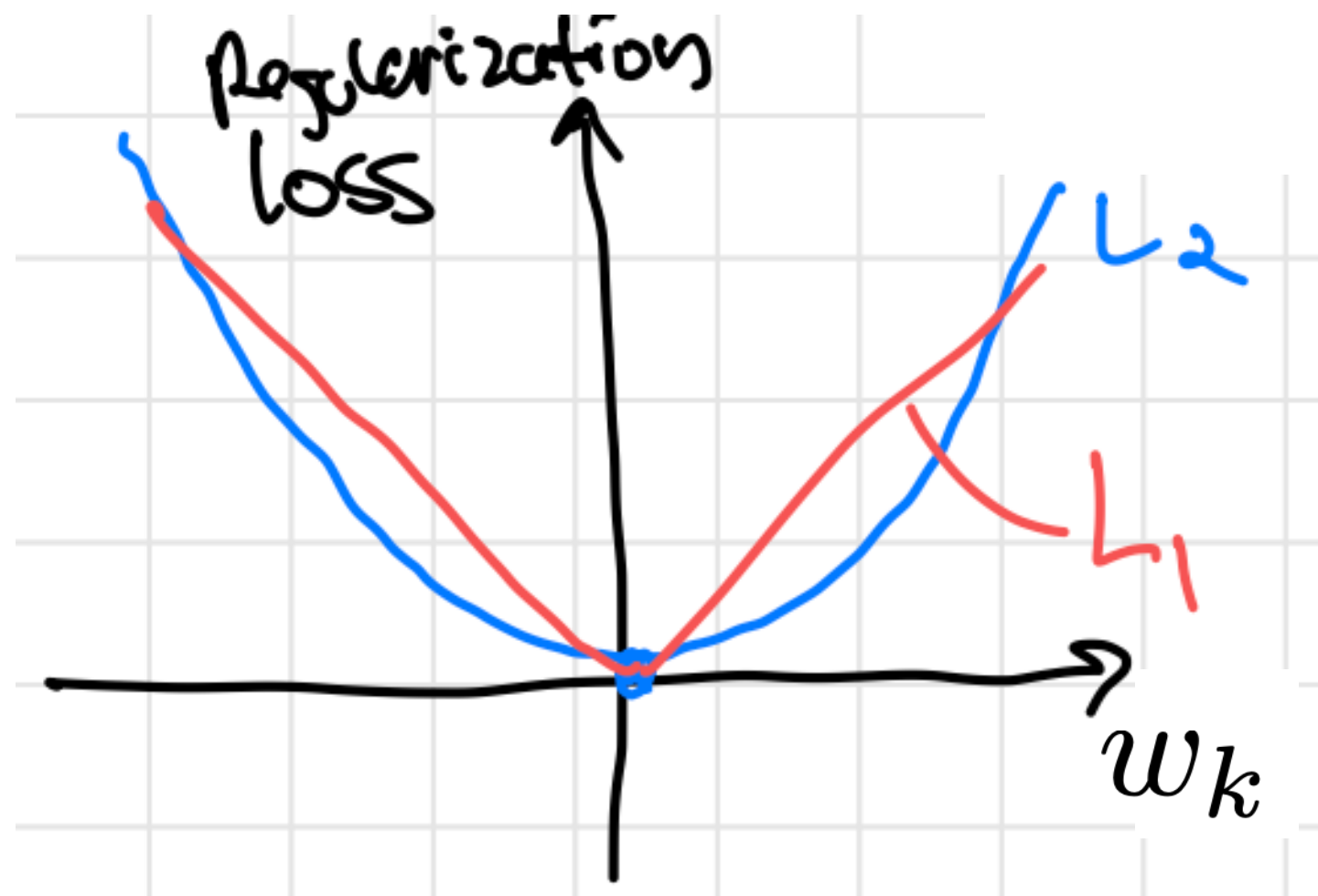
$$w \leftarrow w - \eta([\text{old gradient}] + \lambda 2w)$$

Regularization - L1

- L1 Regularization: similar to L2 reg, but we penalize L1 norm of w

$$\|w\|_1 = \sum_{k=1}^d |w_k|$$

L1 vs. L2 regularization:



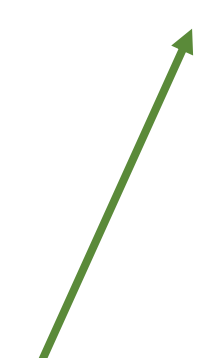
Regularization - L1

- How?

$$\S \quad L(w) = \frac{1}{n} \sum_{i=1}^n (w^\top x^{(i)} - y^{(i)})^2 + \lambda \|w\|_1$$

- What is $\nabla_w \|w\|_1$?

$$\text{sign}(z) \begin{cases} +1, & \text{if } z > 0 \\ -1, & \text{if } z < 0 \\ 0, & \text{if } z = 0 \end{cases}$$

$$\frac{d}{dw_j} \sum_{k=1}^d |w_k| = \frac{d}{dw_j} |w_j| = \text{sign}(w_j)$$


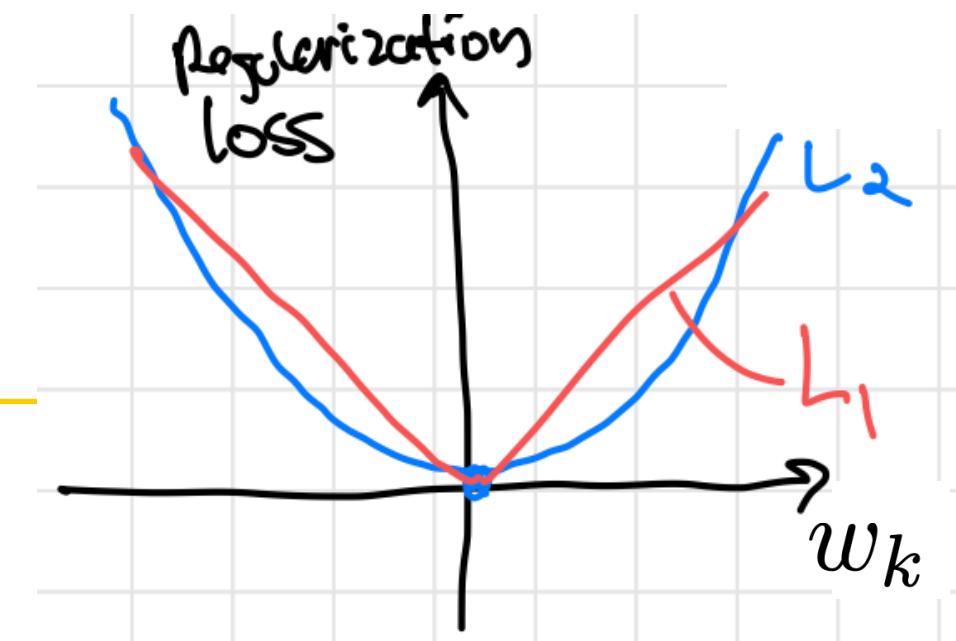
Regularization - L1

- What is $\nabla_w \|w\|_1$?

$$\frac{d}{dw_j} \sum_{k=1}^d |w_k| = \frac{d}{dw_j} |w_j| = \text{sign}(w_j)$$

$$\nabla_w \|w\|_1 = \begin{bmatrix} \text{sign}(w_1) \\ \text{sign}(w_2) \\ \dots \\ \text{sign}(w_d) \end{bmatrix} = \text{sign}(w)$$

Regularization - L1 and L2



$$\nabla_w \|w\|_1 = \begin{bmatrix} \text{sign}(w_1) \\ \text{sign}(w_2) \\ \dots \\ \text{sign}(w_d) \end{bmatrix} = \text{sign}(w)$$

$$\nabla_w \|w\|^2 = 2w$$

- compare with L2 case

encourage “sparse” weights

• L1 always take constant — sized $\longrightarrow$ encourage some w_j 's to be exactly 0

• L2 takes small step for small w_j ; $\longrightarrow$ avoid really big w_j 's and large step for large w_j

Regularization - L1 and L2

$$\nabla_w \|w\|_1 = \begin{bmatrix} \text{sign}(w_1) \\ \text{sign}(w_2) \\ \dots \\ \text{sign}(w_d) \end{bmatrix} = \text{sign}(w) \qquad \nabla_w \|w\|^2 = 2w$$

- Takeaway:
 - L1 learn w_j 's = 0 $\rightarrow$ ignore feature j ; so we learn to **select** only useful features